The basic mechanism of subclassing is that the child inherits all of the capabilities of its parent class. All the parent's instance methods are available to instances of the child.

Let's look at a minimal example and then later build on it. Here's a definition of a parent class and a child class that inherits from it:

```
class Parent
  def say_hello
    puts "Hello from #{self}"
  end
end

p = Parent.new
p.say_hello

class Child < Parent
end

c = Child.new
c.say_hello
```

produces:

```
Hello from #<Parent:0x0000000104796440>
Hello from #<Child:0x00000001047960a8>
```

The parent class defines a single instance method, say_hello. We call that method by creating a new instance of the class and store a reference to that instance in the variable p, and then using dot syntax, p.say_hello.

We then create a subclass using class Child < Parent. The < notation means we're creating a subclass of the thing on the right; the fact that we use less-than is meant to signal that the child class is supposed to be a specialization of the parent.

Note that the child class defines no methods, but when we create an instance of it, we can call say_hello. That's because the child inherits all the methods of its parent. Note also that when we output the value of self—the current object—it shows that we're in an instance of class Child, even though the method we're running is defined in the parent.

The superclass method returns the parent of a particular class:

```
class Parent
end

class Child < Parent
end

Child.superclass # => Parent
```

But what's the superclass of Parent?

```
class Parent
end
Parent.superclass # => Object
```

If you don't define an explicit superclass when defining a class, Ruby automatically uses built-in class Object as class's parent. Let's go further:

```
Object.superclass # => BasicObject
```